

Перкевич Ярослав, IT-32
Практична робота 5

Завдання 1.

Я написав функцію `print_card()` без параметрів, яка друкує моє ім'я, прізвище, групу та рік народження. Викликав її тричі.

Нижче написав таку саму функцію але з параметрами і викликав її трьома різними способами: лише позиційними аргументами, лише іменованими і змішано. Потім створив функцію де параметр групи був за замовчуванням.

```
task1.py > ...
1  # Завдання 1, Перкевич, IT-32
2
3  print('Yaroslav Perkevych, IT-32')
4
5  # Без параметрів
6  def print_card():
7      print(
8          'Name: Yaroslav.\n'
9          'Surname: Perkevych.\n'
10         'Group: IT-32.\n'
11         'Year: 2008'
12     )
13
14  print('no parameters, call 1')
15  print_card()
16
17  print('no parameters, call 2')
18  print_card()
19
20  print('no parameters, call 3')
21  print_card()
22
23  # 3 параметрами
24  def print_card_args(name, surname, group, year):
25      print(
26          f'Name: {name}\n'
27          f'Surname: {surname}\n'
28          f'Group: {group}\n'
29          f'Year: {year}'
30      )
31
32  print('positional arguments')
33  print_card_args('Yaroslav', 'Perkevych', 'IT-32', 2008)
34
35  print('keyword arguments')
36  print_card_args(surname='Perkevych', year=2008, name='Yaroslav', group='IT-32')
37
38  print('mixed arguments')
39  print_card_args('Yaroslav', 'Perkevych', year=2008, group='IT-32')
40
```

```
# Зі значенням за замовчуванням

def print_card_default(name, surname, year, group='IT-32'):
    print(
        f'Name: {name}\n'
        f'Surname: {surname}\n'
        f'Group: {group}\n'
        f'Year: {year}'
    )

print('default argument call')
print_card_default('Yaroslav', 'Perkevych', 2008)
```

Результат:

```
admin@Yaroslavs-MacBook-Air pr5 % /usr/bin/python3
ограмування/pr5/task1.py
Yaroslav Perkevych, IT-32
no parameters, call 1
Name: Yaroslav.
Surname: Perkevych.
Group: IT-32.
Year: 2008
no parameters, call 2
Name: Yaroslav.
Surname: Perkevych.
Group: IT-32.
Year: 2008
no parameters, call 3
Name: Yaroslav.
Surname: Perkevych.
Group: IT-32.
Year: 2008
positional arguments
Name: Yaroslav
Surname: Perkevych
Group: IT-32
Year: 2008
keyword arguments
Name: Yaroslav
Surname: Perkevych
Group: IT-32
Year: 2008
mixed arguments
Name: Yaroslav
Surname: Perkevych
Group: IT-32
Year: 2008
default argument call
Name: Yaroslav
Surname: Perkevych
Group: IT-32
Year: 2008
```

Спробував викликати `print_card_args("Yaroslav")`.

```
54 print_card_args('Yaroslav')
```

Видало помилку:

```
Traceback (most recent call last):
  File "/Users/admin/Desktop/Всяке/Програмування/pr5/task1.py", line 54, in <module>
    print_card_args('Yaroslav')
TypeError: print_card_args() missing 3 required positional arguments: 'surname', 'group', and 'year'
```

Спробував вивести:

```
55 print_card_args(name='Yaroslav', 'Perkevych')
```

Ще на етапі написання редактор коду став підсвічувати помилку. А при запуску висвітлюється така помилка:

```
admin@Yaroslav-MacBook-Air-pr5: ~/Desktop/Всяке/Програмування/pr5 % /usr/bin/python3 /Users/admin/Desktop/Всяке/Програмування/pr5/task1.py
File "/Users/admin/Desktop/Всяке/Програмування/pr5/task1.py", line 55
    print_card_args(name='Yaroslav', 'Perkevych')
                                ^
SyntaxError: positional argument follows keyword argument
```

Завдання 2.

Я написав дві функції: перша виводить вік і нічого не повертає, а друга повертає вік. Якщо в другій рік народження менший за нуль, або більший за поточний рік то повертає -1.

```

task2.py > ...
1  # Завдання 2, Перкевич, IT-32
2  print('Yaroslav Perkevych, IT-32')
3
4  def print_age(year):
5      age = 2026 - year
6      print(age)
7
8  def get_age(year, current_year=2026):
9      if year < 0 or year > current_year:
10         return -1
11     else:
12         age = current_year - year
13         return age
14         print("after return")
15
16  print_age(2008)
17  get_age(2008)
18  print(print_age(2008))
19

```

Результат:

```

admin@Yaroslav-MacBook-Air pr5
/pr5/task2.py
Yaroslav Perkevych, IT-32
18
18
None
admin@Yaroslav-MacBook-Air pr5

```

Коли виконується рядок `print(print_age(2008))`, то спочатку виконується функція всередині дужок і виводиться 18, а потім ми бачимо, що оскільки функція не повертає нічого, то за замовчуванням повертає значення `None`, яке є результатом виводу. Потім, використовуючи результат `get_age` я порахував свій вік в місяцях, тижнях, а також скільки років мені буде в 2030.

```

20 age_in_months = get_age(2008) * 12
21 print(f'Age in month: {age_in_months}')
22
23 age_in_weeks = get_age(2008) * 52
24 print(f'Age in weeks: {age_in_weeks}')
25
26 age_in_2030 = get_age(2008, current_year=2030)
27 print(f'Age in 2030: {age_in_2030}')
28

```

```

Age in month: 216
Age in weeks: 936
Age in 2030: 22

```

Потім я спробував зробити схожу дію з print_age:

```
print(print_age(2008) * 12)
```

Отримали помилку:

```

Traceback (most recent call last):
  File "/Users/admin/Desktop/Всяке/Програмування/pr5/task2.py", line 29, in <module>
    print(print_age(2008) * 12)
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
admin@Yaroslavs-MacBook-Air pr5 %

```

Наприкінці я зробив перевірку рядка `if year < 0 or year > current_year:` який міститься в функції `get_age`, зі значенням 3000:

```

30
31 print(f'Check at a value of 3000: {get_age(3000)}')

```

Все працює як і очікувалось:

```

Age in 2030: 22
Check at a value of 3000: -1
admin@Yaroslavs-MacBook-Air pr5 %

```

Після return в `get_age` я дописав `print("after return")`, але цей рядок не виконується, бо return повертає значення і миттєво закінчує виконання функції.

Завдання 3.

Я створив функцію з анотацією, яка за допомогою індексів бере перші букви з імені та прізвища, переводить їх у верхній регістр і

склеює через крапку.

```
task3.py > count_vowels
1 # Завдання 3, Перкевич, IT-32
2 print('Yaroslav Perkevych, IT-32')
3
4 def get_initials(name: str, surname: str) -> str:
5     return f"{name[0].upper()}.{surname[0].upper()}."
6
```

Наступна функція повертає кількість входжень letter у text без урахування регістру. всередині має докстрінг.

```
def count_letters(text: str, letter: str = "a") -> int:
    """Return how many times letter occurs in text."""
    return text.lower().count(letter.lower())
```

Третя функція повертає кількість голосних

```
def count_vowels(text: str) -> int:
    vowels = "aeiouy"
    count = 0
    for char in text.lower():
        if char in vowels:
            count += 1
    return count
```

Остання функція повертає рядок у зворотньому порядку(бере букву за буквою, і приліплює кожен наступний зліва, так і виходить перевернутий рядок).

```
def reverse_text(text: str) -> str:
    reversed_str = ""
    for char in text:
        reversed_str = char + reversed_str
    return reversed_str
```

Далі задаю в змінні свої дані, виводжу своє прізвище та ім'я, потім ініціали, довжину прізвища, кількість голосних та приголосних. потім кількість кожної голосної, перевернуте прізвище, докстрінг і анотації

```
28
29     name = "Yaroslav"
30     surname = "Perkevych"
31     group = "IT-32"
32
33     print(f"{name} {surname}, {group}")
34     print(f"Initials: {get_initials(name, surname)}")
35
36     c = len(surname)
37     vowels = count_vowels(surname)
38     consonants = c - vowels
39
40     print(f"Letters in surname: {c}")
41     print(f"Vowels: {vowels}, consonants: {consonants}")
42
43     for v in "aeiou":
44         print(f"{v}: {count_letters(surname, letter=v)}")
45     ✨
46     print(f"Default letter 'a': {count_letters(surname)}")
47     print(f"Reversed surname: {reverse_text(surname)}")
48     print(f"Docstring: {count_letters.__doc__}")
49     print(f"Annotations: {count_letters.__annotations__}")
```

```
admin@Yaroslavs-MacBook-Air pr5 % /usr/bin/python3 /Users/admin/Desktop/Всяке/Програмування/pr5/task3.py
Yaroslav Perkevych, IT-32
Yaroslav Perkevych, IT-32
Initials: Y.P.
Letters in surname: 9
Vowels: 3, consonants: 6
a: 0
e: 2
i: 0
o: 0
u: 0
Default letter 'a': 0
Reversed surname: hcyvekrep
Docstring: Return how many times letter occurs in text.
Annotations: {'text': <class 'str'>, 'letter': <class 'str'>, 'return': <class 'int'>}
admin@Yaroslavs-MacBook-Air pr5 %
```

Завдання 4

Я побудував програму повністю з функцій:

`read_grade(prompt)` запитує оцінку в користувача доки не введене значення від 0 до 100. і повертає її.

`to_letter(grade)` повертає буквену оцінку.

`average(grades)` повертає середнє арифметичне.

`count_above(grades, limit)` повертає скільки оцінок більші за ліміт

`print_report(name, group, grades)` друкує звіт: список оцінок, середнє з двома знаками після коми та буквену оцінку, найкращий і найгірший бали, кількість оцінок вище середнього.
`main()` задає ім'я та групу, читає кількість оцінок і викликає `print_report`

task4.py > ...

```
1  # Завдання 4, Перкевич, IT-32
2  print('Yaroslav Perkevych, IT-32')
3
4  def read_grade(prompt):
5      """Prompt user until a valid integer grade between 0 and 100 is entered."""
6      while True:
7          value = input(prompt)
8          if not value.isdigit():
9              print("Error: digits only")
10             continue
11             grade = int(value)
12             if 0 <= grade <= 100:
13                 return grade
14             print("Error: the value must be between 0 and 100")
15
16  def to_letter(grade):
17      """Convert numeric grade to letter grade A-F using multiple returns."""
18      if grade >= 90:
19          return "A"
20      if grade >= 82:
21          return "B"
22      if grade >= 74:
23          return "C"
24      if grade >= 64:
25          return "D"
26      if grade >= 60:
27          return "E"
28      return "F"
29
30  def average(grades):
31      """Calculate and return arithmetic mean of grades list."""
32      return sum(grades) / len(grades)
33
34  def count_above(grades, limit):
35      """Count how many grades are strictly greater than the limit."""
36      count = 0
37      for grade in grades:
38          if grade > limit:
39              count += 1
40      return count
41
```

```

41
42 def print_report(name, group, grades):
43     """Print complete academic report using helper functions."""
44     avg_grade = average(grades)
45     letter = to_letter(avg_grade)
46     above_avg = count_above(grades, avg_grade)
47
48     print("--- Report ---")
49     print(f"Student: {name}, group {group}")
50
51     print("Grades:", end=" ")
52     for g in grades:
53         print(g, end=" ")
54     print()
55
56     print(f"Average: {avg_grade:.2f} -> {letter}")
57     print(f"Best: {max(grades)}, worst: {min(grades)}")
58     print(f"Above average: {above_avg}")
59
60 def main():
61     """Run the main grading workflow."""
62     name = "Yaroslav Perkevych"
63     group = "IT-32"
64     first_name = "Yaroslav"
65     n = max(3, len(first_name))
66
67     grades = []
68     for i in range(1, n + 1):
69         grade = read_grade(f"Grade {i} (0-100): ")
70         grades.append(grade)
71
72     print_report(name, group, grades)
73
74     main()

```

Якщо порівнювати з завданням з практичної 4, то з використанням функцій(коли кожна функція виконує одну дію) код більш структурований, читабельний, а також потім за потреби даний код буде легше виправляти.

Результат:

```
● admin@Yaroslavs-MacBook-Air pr5 % /usr/bin/python3 /Users/a  
/pr5/task4.py  
Yaroslav Perkevych, IT-32  
Yaroslav Perkevych, IT-32  
Grade 1 (0-100): 95  
Grade 2 (0-100): 90  
Grade 3 (0-100): 80  
Grade 4 (0-100): 92  
Grade 5 (0-100): 90  
Grade 6 (0-100): 96  
Grade 7 (0-100): 85  
Grade 8 (0-100): 88  
--- Report ---  
Student: Yaroslav Perkevych, group IT-32  
Grades: 95 90 80 92 90 96 85 88  
Average: 89.50 -> B  
Best: 96, worst: 80  
Above average: 5  
○ admin@Yaroslavs-MacBook-Air pr5 % █
```